

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	y.hema sai lahari	Reg. No.:	192311446
Date:	20-08-2026	Duration:	60 minutes

Code of Conduct

I y.hema sai lahari(192311446) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: y.hema sai lahari

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.

- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. Commit History Analysis

- Present the commit history of the project.
- Explain how commit messages follow good version control practices and contribute to project maintenance.

7. Docker Environment Design

- Explain why Docker is suitable for the assigned project.
- Identify the application components that require containerization.

8. Dockerfile Development

- Design and explain the Dockerfile required to containerize the application.
- Describe the purpose of each instruction used in the Dockerfile.

9. Docker Compose Design

- Develop a Docker Compose configuration for the project.
- Explain the services, networking, volumes, environment variables, and dependencies used.

10. Container Deployment and Testing

- Demonstrate the execution of the application using Docker containers.
- Explain the testing procedure and provide evidence that the application runs successfully.

11. Benefits of Git and Docker

- Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

12. Challenges and Improvements

- Identify the challenges encountered during implementation.
- Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

	proper repository organization.			
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots,	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

	references, discussion of benefits, challenges, and future improvements.			
--	---	--	--	--

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Project Title

Temple Donation and Event Management System

1. Problem Analysis

1.1 Problem Statement

The **Temple Donation and Event Management System** is a web-based application designed to digitally manage temple donations, devotees, religious events, and donation records. Traditional temple donation management may involve manual registers, paper receipts, and separate records for different events. This can lead to data duplication, difficulty in tracking donations, delays in generating reports, and difficulty in maintaining accurate records.

The proposed system provides a centralized platform where administrators can manage devotees, record donations, organize temple events, and generate donation-related reports. The system improves transparency, accuracy, accessibility, and efficiency in temple administration.

Project Objectives

The major objectives of the system are:

1. To maintain digital records of temple devotees and donors.
2. To record and manage online and offline donations.
3. To maintain information about temple events and festivals.
4. To provide administrators with a centralized dashboard.
5. To generate donation and event reports.
6. To reduce manual paperwork and data duplication.
7. To improve transparency and accuracy in donation management.
8. To provide a reliable and scalable application using Docker.

Functional Requirements

Admin Module

- Admin login and authentication.
- Add, update, and delete donor information.
- Manage donation records.
- Manage temple events.
- View donation history.
- Generate reports.

Donor Module

- Donor registration.

- View donation details.
- Make donation entries.
- View donation history.
- Receive donation confirmation/receipt.

Event Management Module

- Create temple events.
- Update event details.
- Display upcoming events.
- Maintain event schedules.
- Manage event-related donation information.

Reporting Module

- Generate daily, monthly, and yearly donation reports.
- View total donations.
- Filter donations by donor, date, or event.
- Generate downloadable reports.

Expected Outcomes

The expected outcome is a secure and user-friendly system that simplifies temple donation and event management. It should reduce manual work, improve record accuracy, provide quick access to donation information, and support efficient temple administration.

2. Project Structure Design

2.1 Repository Folder Structure

The project repository can be organized as follows:

Temple-Donation-Management/

```

|
|— frontend/
|  |— public/
|  |— src/
|   |— components/
|   |— pages/
|   |— services/

```

```
| | | └── assets/
| | └── App.js
| └── package.json
| └── Dockerfile
|
└── backend/
    | └── controllers/
    | └── models/
    | └── routes/
    | └── middleware/
    | └── config/
    | └── server.js
    | └── package.json
    | └── Dockerfile
    |
    └── database/
        | └── init/
        |
        └── docs/
            | └── architecture/
            | └── screenshots/
            |
            └── .gitignore
            └── docker-compose.yml
            └── README.md
            └── .env
```

2.2 Purpose of Major Folders and Files

frontend/ – Contains the user interface of the application.

backend/ – Contains server-side logic, APIs, authentication, and database operations.

components/ – Contains reusable user interface components.

pages/ – Contains application pages such as Login, Dashboard, Donations, Donors, and Events.

controllers/ – Handles application business logic.

models/ – Defines database models such as Donor, Donation, and Event.

routes/ – Defines API endpoints.

middleware/ – Contains authentication and request-processing middleware.

config/ – Contains configuration files such as database configuration.

database/ – Contains database initialization scripts.

docs/ – Stores project documentation, architecture diagrams, and screenshots.

Dockerfile – Contains instructions to create the application Docker image.

docker-compose.yml – Defines and manages multiple application services.

.gitignore – Specifies files that should not be tracked by Git.

README.md – Provides project description, installation steps, commands, and usage information.

.env – Stores environment-specific configuration such as database credentials and ports. It should not be committed to Git.

3. Git Repository Initialization

3.1 Repository Creation

Git is used to track changes in the Temple Donation and Event Management System.

The following commands can be used to initialize the repository:

```
mkdir Temple-Donation-Management
```

```
cd Temple-Donation-Management
```

```
git init
```

```
git status
```

```
git add .
```

```
git commit -m "Initial commit - Temple Donation Management System"
```

A remote repository can then be connected using:

```
git remote add origin <repository-url>
```

```
git branch -M main
```

```
git push -u origin main
```

3.2 Essential Git Files

.git/

The .git directory contains Git's internal information, including commit history, branches, configuration, and object database.

.gitignore

The .gitignore file prevents unnecessary or sensitive files from being committed.

Example:

node_modules/

.env

dist/

build/

*.log

README.md

Contains project information, setup instructions, technologies used, and Git/Docker commands.

Git Configuration

Git can be configured using:

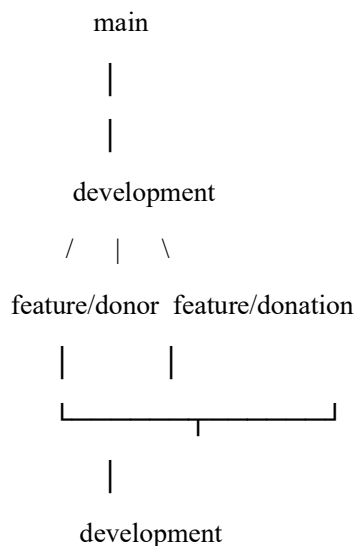
```
git config --global user.name "Your Name"
```

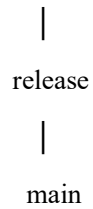
```
git config --global user.email "your@email.com"
```

4. Git Branching Strategy

4.1 Selected Branching Model

The project uses a simplified **Git Flow branching strategy**.





Main Branch

The main branch contains stable and tested versions of the project.

Development Branch

The development branch contains integrated development work before it is released.

Feature Branches

Feature branches are created for individual functionalities.

Examples:

feature/donor-management

feature/donation-management

feature/event-management

feature/report-generation

Release Branch

A release branch is created when the application is ready for final testing and deployment.

Hotfix Branch

A hotfix branch is created to fix critical problems in the production version.

4.2 Justification

This branching strategy is suitable because the project contains multiple modules that can be developed independently. Developers can work on different features without directly modifying the stable main branch. After testing, features are merged into the development branch and finally released to the main branch.

5. Version Control Workflow

The Git workflow followed for the project is:

Create Repository



Create Development Branch



Create Feature Branch

↓
Develop Feature
↓
git add
↓
git commit
↓
Merge Feature
↓
Testing
↓
Release
↓
Merge to Main
↓
Push to Remote Repository

Git Commands

Check Repository Status

git status

Create Development Branch

git checkout -b development

Create Feature Branch

git checkout -b feature/donation-management

Add Files

git add .

Commit Changes

git commit -m "Add donation management module"

Switch Branch

git checkout development

Merge Feature

git merge feature/donation-management

Push Changes

`git push origin development`

Pull Latest Changes

`git pull origin development`

Conflict Resolution

If two branches modify the same portion of a file, Git may report a merge conflict.

The conflict can be resolved manually and then committed:

`git status`

After correcting the file:

`git add .`

`git commit -m "Resolve merge conflict"`

Git operations help maintain a controlled development history and allow developers to collaborate without losing previous versions.

6. Commit History Analysis

The commit history can be viewed using:

`git log --oneline --graph --all`

Example commit history:

a81d32f Add Docker Compose configuration

b72c19a Add event management module

c63f41b Add donation management module

d52e18a Add donor management module

e41a27c Create backend API

f30b16d Initial project structure

Good Commit Message Practices

Commit messages should clearly describe the change made.

Examples:

Initial project structure

Add donor registration module

Add donation management API

Add temple event management

Update donation validation

Add Docker configuration

Fix database connection issue

Update README documentation

Good commit messages make the project history easy to understand. They help developers identify when a feature was added, when a problem was fixed, and how the project evolved.

7. Docker Environment Design

7.1 Why Docker is Suitable

Docker provides a consistent environment for developing, testing, and deploying the Temple Donation and Event Management System.

Without Docker, different developers may have different versions of Node.js, databases, or dependencies. Docker packages the application and its dependencies into containers, reducing environment-related problems.

Benefits

- Consistent development environment.
- Easy application deployment.
- Dependency isolation.
- Portability between systems.
- Easy scaling.
- Faster setup.
- Simplified testing.
- Better maintainability.

7.2 Application Components Requiring Containerization

The main components are:

Frontend Container

|

↓

Backend/API Container

|

↓

Database Container

The frontend provides the user interface, the backend processes requests and business logic, and the database stores donors, donations, events, and related records.

8. Dockerfile Development

8.1 Backend Dockerfile

Example:

```
FROM node:20
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["npm", "start"]
```

Explanation of Dockerfile Instructions

FROM

```
FROM node:20
```

Specifies the base Node.js image.

WORKDIR

```
WORKDIR /app
```

Sets /app as the working directory inside the container.

COPY

```
COPY package*.json ./
```

Copies package configuration files into the container.

RUN

```
RUN npm install
```

Installs the required application dependencies.

COPY

COPY . .

Copies the remaining application source code.

EXPOSE

EXPOSE 5000

Documents the port used by the backend application.

CMD

CMD ["npm", "start"]

Starts the backend application when the container runs.

9. Docker Compose Design

The docker-compose.yml file can be used to manage the frontend, backend, and database together.

Example:

services:

frontend:

build: ./frontend

ports:

- "3000:3000"

depends_on:

- backend

backend:

build: ./backend

ports:

- "5000:5000"

environment:

- DB_HOST=database

- DB_PORT=5432

- DB_NAME=templadb

- DB_USER=admin
- DB_PASSWORD=admin

depends_on:

- database

database:

image: postgres:16

environment:

POSTGRES_DB: templedb

POSTGRES_USER: admin

POSTGRES_PASSWORD: admin

ports:

- "5432:5432"

volumes:

- temple_data:/var/lib/postgresql/data

volumes:

temple_data:

Services

Frontend

Runs the user interface of the system.

Backend

Runs the API and application logic.

Database

Stores donor, donation, event, and user information.

Networking

Docker Compose automatically creates a network allowing containers to communicate using service names.

For example:

backend → database:5432

Volumes

The `temple_data` volume provides persistent database storage. Database information is preserved even if the database container is recreated.

Environment Variables

Environment variables are used to configure database connection information without hardcoding configuration into the source code.

Dependencies

The `depends_on` option ensures that dependent services are started in the appropriate order.

10. Container Deployment and Testing

10.1 Build and Start Containers

The complete application can be started using:

```
docker compose up --build
```

To run containers in the background:

```
docker compose up -d
```

10.2 Check Running Containers

```
docker ps
```

Expected output should show containers for:

```
frontend
```

```
backend
```

```
database
```

10.3 View Container Logs

```
docker compose logs
```

For backend logs:

```
docker compose logs backend
```

10.4 Stop Containers

```
docker compose down
```

Testing Procedure

The application can be tested using the following procedure:

1. Start all containers using Docker Compose.
2. Verify that all containers are running.
3. Open the frontend application in a web browser.

4. Test admin login.
5. Add a donor.
6. Add a donation.
7. Create a temple event.
8. Verify that donation details are stored in the database.
9. Check donation history and reports.
10. Verify that frontend and backend communicate correctly.

Evidence/Screenshots to Include

Screenshot 1: Git repository initialization.

Screenshot 2: git status output.

Screenshot 3: Branch creation.

Screenshot 4: Git commit history.

Screenshot 5: Docker image creation.

Screenshot 6: docker compose up --build.

Screenshot 7: docker ps showing running containers.

Screenshot 8: Application running in browser.

Screenshot 9: Donation entry screen.

Screenshot 10: Event management screen.

11. Benefits of Git and Docker

Git Benefits

Collaboration

Git allows multiple developers to work on different features simultaneously.

Version Management

Git maintains a complete history of project changes.

Branching

Different features can be developed independently using branches.

Backup

A remote Git repository provides a backup of the source code.

Error Recovery

Previous versions can be restored when necessary.

Docker Benefits

Portability

The application can run consistently on different systems.

Deployment

Docker simplifies application deployment.

Isolation

Each service runs in its own isolated environment.

Scalability

Additional containers can be created when application demand increases.

Maintainability

Application services and dependencies can be managed separately.

Combined Benefits

Using Git and Docker together provides a reliable development workflow:

Git

↓

Version Control

↓

Source Code

↓

Docker

↓

Containerization

↓

Testing

↓

Deployment

Git manages the source code and its history, while Docker provides a consistent environment for running the application.

12. Challenges and Improvements

12.1 Challenges Encountered

Git Merge Conflicts

Changes made by multiple developers to the same file can cause merge conflicts.

Dependency Issues

Different versions of Node.js or packages may cause compatibility problems.

Docker Configuration

Correctly configuring ports, environment variables, networking, and volumes can be challenging.

Database Connectivity

The backend must correctly connect to the database container using the Docker service name.

Environment Configuration

Sensitive values such as database passwords should be handled securely using environment variables.

Container Debugging

Errors inside containers may require checking logs and container status.

12.2 Possible Improvements

1. Implement automated GitHub Actions CI/CD.
2. Add automated unit and integration testing.
3. Use Docker multi-stage builds to reduce image size.
4. Use secure environment variable management.
5. Add database backup and recovery mechanisms.
6. Implement role-based access control.
7. Add online payment gateway integration.
8. Add email/SMS donation confirmation.
9. Add QR-code-based donation receipts.
10. Add analytics dashboards for donation trends.
11. Implement monitoring and logging.
12. Deploy the containers using a cloud platform.

Conclusion

The **Temple Donation and Event Management System** provides a digital solution for managing temple donors, donations, events, and reports. Git provides effective version control, collaboration, branching, and change tracking, while Docker provides a consistent and portable environment for application deployment.

The combination of Git and Docker improves the overall development lifecycle by making the system easier to develop, test, maintain, and deploy. The proposed architecture can also be extended in the future with online payment integration, automated reports, cloud deployment, analytics, and CI/CD pipelines.

Therefore, Git and Docker provide a strong technical foundation for developing and deploying a reliable Temple Donation and Event Management System.